NPC

General Theory Refresher

Runtimes

1. Polynomial
    - O(n^k) for some constant k
2. Exponential
    - O(k^n) for some k > 1
3. Pseudo-polynomial
    - Polynomial to the magnitude of the input, but exponential to the size of the input
    - Examples include Knapsack O(nB) and Ford-Fulkerson O(mC)

A bit more on whatever “pseudo-polynomial” means.

- Note that n and m of Knapsack and Ford-Fulkerson runtimes are based on the *size* of the inputs (n objects, m edges). However, B and C are **magnitudes**. Magnitudes are a single number that increase the runtime without needing more space.
- Suppose that n is represented by two 32-bit numbers. Then, each increment of n scales in a predictable manner.

1 item	64 bits
2 items	128 bits (2 * 64)
3 items	192 bits (3 * 64)

- On the other hand, B and C are just numbers (magnitude). Consider how the magnitude (left) and the space required for representation (right) scales:

B=1	1 bit
B=2	2 bits
B=3	2 bits
B=4	3 bits
B=5	3 bits
B=6	3 bits
B=7	3 bits
B=8	4 bits
B=x	log_2(x) bits

- Tracking the linear increases in the number of bits (size of input), we get that the magnitude of B increases at 1, 2, 4, and then 8! This is what is meant by *exponential to the size of the input*.
- Of course, if we track the increase in magnitude, then it appears linear. This is what is meant by *polynomial to the magnitude of the input*.
- Any runtime based on log(B) is considered polynomial (this is important!)

- Perhaps it is more helpful to note that pseudo-polynomial runtimes act like exponential runtimes, but it is called pseudo-polynomial as there is no explicit exponent in the Big-Oh notation.
    - Then if we ignore the magnitude B and instead choose to represent it as 2^k where k is the number of bits, it is easy to see that O(nk^2) is indeed exponential.

Reductions

All problems provided will be NP-complete; there are no gotchas. Just worry about the reduction.

Reductions are from **known to unknown**.

- The best way that I understood this was to assume that the unknown problem can be solved by some magical artifact. Then, we try to solve existing problems that are known to be NP-complete with this magical artifact.
- Technically the reductions can go both ways (as both problems are in NP-complete). However, we want to show that this unknown/new/given problem is at least as hard as the known NP-complete problems. In other words, if we have the magical artifact, we can also solve the known NP-complete problem.

NP-Complete Proofs

Given a problem B:

1. Pick a known problem A to reduce to problem B

2. Demonstrate that B is in the class NP
    - Give steps on how to verify a candidate solution for B in polynomial time.
        - Prove all requirements for the solution (a solution might be incorrect), but do not worry about invalid/junk solutions (e.g., assume correct data types and consistent variables)
    - Give the runtime for verification in Big-Oh notation.
        - Just saying "this is polynomial" is not enough
        - **Do not** express this in terms of O(k), O(b), or O(g), where k, b, and g are goals/targets unless you bind it. Note that these are magnitudes and therefore make the runtime pseudopolynomial. Just use O(n), or if you really want to, state that k <= n.

3. Demonstrate that problem B is at least as hard as the chosen problem in A.
    - Show how an instance of A can be converted to an instance of B in polynomial time. This must be shown for *all* instances of A. In other words, we cannot simply avoid instances of problem A.
        - The point is to force a solution to B if there is a solution to A.
        - Again, give the runtime for input transformation in Big-Oh notation.
        - Never solve A to create an instance for B (firstly that's kinda stupid because solving A is likely not polynomial)
    - Show how a solution to B can be converted to a solution to A in polynomial time.
        - All solutions of B must convert to a solution of A.
        - If B has no solution, then A also has no solution. Just regurgitate this in the exam, as this costs points.
        - Again, give the runtime for input transformation in Big-Oh notation.

4. Show that B has a solution if and only if A has a solution.
    - Basically, prove A -> B and B -> A. Use the contrapositives (~B -> ~A and ~A -> ~B respectively) wherever possible.
    - Do not accidentally argue the same direction twice.

Tenets

1. We are going from known problem A to unknown problem B
2. We don’t know how the unknown problem can be solved. We don’t care either.
3. We never try to solve the known problem ourselves. This makes the runtime non-polynomial. We just hand it over to the magical black-box that solves the unknown problem.
4. We do not assume that the input to problem A has a solution.
5. We make sure that the solution to B is transformed back to the solution for A. If no transformation is required, we explicitly state this.
6. We make sure to explicitly state that if B has no solution, neither does A. We then say that this (rather inane statement) takes O(1) time.
7. We always give runtimes in Big-Oh notation

Strategies

SAT Reductions

Examples include: 3SAT to Exact 4 SAT and SAT to Stingy SAT

NP Proof

- Make sure to verify SAT assignments
- Make sure to count everything
- Make sure to verify any special conditions in the solution

Input

- Be sure to maintain the CNF
- Be sure to maintain equivalent logic
- Be sure to only add literals and clauses. **Never** add hardcoded True/False values to anything.

Output

- Remove added literals/variables
- There is no need to remove added clauses as clauses are not part of the output. Recall that SAT outputs literal/variable assignments

Correctness

- Establish equivalency of the two SAT problems after transformation
- Then argue if-and-only-if

Graph Reductions

Connectivity

Examples include IS to Clique + IS and Clique to Subgraph Isomorphism

NP Proof

- Make sure to verify the graph carefully (duh)
- If the graph contains a clique but isn't separated, you must separate it first.
    - It is possible that the graph is a clique + something else. Remove that something else then check for the presence of a clique, as finding a clique in an arbitrary graph is not polynomial.
- Make sure to count anything or check the size of the solutions.
- Try to avoid degree counting as that is usually wrong or is much harder.
    - I think it is more accurate to suggest avoiding using some formula that utilises node degrees.

- On the other hand, feel free to use degrees to delete/focus specific nodes. For example, when looking for a k-clique, any nodes with degree < k-1 can be dropped.

Input

- As usual, do not even attempt to solve the problem.
- Remember your b, k, or g (targets)
- Be mindful of accidentally adding extra candidate vertex/edges. More concretely, do not accidentally add such that a solution is created where there should be none.

Output

- Isolate the desired output. Often just removing your added vertices/edges is not enough.

Pathing

Examples include Rudrata Cycle to TSP (with a budget) and Rudrata Path to Spanning Tree with $L \subseteq V$ leaves.

NP Proof

- Make sure to verify the path correctly
- Make sure that all vertices are accounted for
- Make sure that the path is a path, the cycle is a cycle, or the spanning tree is a tree
- Make sure to count anything or check the size of the solutions
- Use the graph black boxes

Input

- Consider what each Rudrata problem is looking for, and try all of them if you have to.
- Consider forcing paths to a cycle
- Consider forcing leaves to be S and T
- Do you even need an S and T?

Output

- Generally pretty simple, but make sure to transform to a path/cycle if you have to
- Even if there is no transformation, state it and give a O(1) runtime.

Set Reductions

An example would be vertex cover to hitting set.

NP Proof

- Make sure to verify each set and its contents or intersects
- Make sure to count anything or check the size of sets
- Consider runtimes in how large the largest set can be (usually the size of all allowed items)

Input

- Only use allowed NP-C problems for reduction (duh)
- It is possible to start from a graph or SAT, but be careful with SAT as literals are related (a and ~a are related). In sets, there is no such thing as a relationship.
- This is usually a generalisation problem; that is, create an equivalent problem to be solved.

Output

- Nothing special

Correctness

- Argue why both sets are equivalent
- Then argue if-and-only-if

Summation Reductions

An example would be Subset Sum to Knapsack

NP Proof

- Generally straightforward to verify. Does the solution set add up to what we are looking for?
- Adding to some magnitude for verification is polynomial due to O(n log X) time, where X is the sum of what you are adding. No idea what the fuck this means.

Input

- Force sums to represent the solution
- Consider bit or integer representations
- Consider forcing inequalities to converge on equality

Output/Correctness

- Nothing special

Cross-domain Reductions

Apparently these would be considered too difficult for the exam.

Allowable Problems

Problem	**Input**	**Output**
SAT	A boolean formula in CNF	A satisfying assignment or NO
3SAT	A boolean formula in CNF where each clause has *at most* 3 literals	A satisfying assignment or NO
Clique	An undirected graph and a goal integer k	A set of vertices of size k *where all pairs are connected* if one exists or NO
Independent Set (IS)	An undirected graph and a goal integer k	A set of vertices of size k *where no two are connected by an edge* or NO
Vertex Cover (VC)	An undirected graph and a *budget* integer b	A set of vertices of size <= b that *touches all edges* or NO
Subset Sum (SSS)	Positive integers a_1, …, a_n and a target k	A subset that sums to k or NO
Rudrata Path	An undirected graph	A simple path visiting all vertices exactly once (Hamiltonian path) or NO
Rudrata (s, t)-Path	An undirected graph, vertices s and t	A Hamiltonian path that starts at s and ends at t or NO
Rudrata cycle	An undirected graph	A simple cycle visiting all vertices exactly once or NO
Integer Linear Programming	An objective, constraints, and variables restricted to integers	An integer-feasible optimal solution or NO
Zero-One Equations	Linear equations over variables restricted to {0, 1}	A 0/1 assignment satisfying all equations or NO
3D-Matching	Sets X, Y, and Z of equal size and a set of triples $T \subseteq X \times Y \times Z$	A matching of size |X| using disjoint triples; each element of X, Y, Z appears at most in one triple or NO.
Travelling Salesman (TSP)	Distance matrix D and budget b	A cycle that visits each city exactly once and returns to the start with a total cost <= b or NO

Notes to Self

NP Proof

In general, I think that the input to a checker contains the:

a. Candidate solution
b. All inputs and arguments

It is therefore not just a problem of checking only the solution, because some arguments (e.g., goals and budgets) cannot be captured solely in the solution itself.

Checking for SAT

Given a candidate solution to the input formula:

- Check clause by clause such that each clause is satisfied (has at least one literal set to TRUE). This takes O(nm) time, n = literals and m = clauses.
- Then perform any additional checks (e.g., if at most k literals are TRUE/FALSE). This takes O(n) time

Checking for Clique

Given a candidate solution (hey is this a clique?)

- Check that all pairs of vertices are connected. This takes O(n^2) time.
- Then perform any additional checks (e.g., the number of vertices > goal g). This takes O(n) time.

Given a candidate solution that *might* have a clique (hey is there a clique?)

- This problem is NP-complete, so don't bother trying to perfectly isolate the clique first. Remember the point is to check against the given problem, not to fight for a perfect clique finder.
- Either delete non-clique vertices (might be easier) based on known properties (e.g., degrees, path, etc.)
- Or isolate all vertices with >= k-1 degrees to have an easier time checking. This takes O(n+m) time.

Checking for Subgraph

Given an original graph H, a candidate subgraph G, and a mapping M from G to H.

- Check that all vertices v in G map exactly to one vertex in H. That is, M(v) in H. This takes O(n) time, where n is the number of vertices in G.
- Check that all edges u, v in G map exactly to one edge in H. That is, (M(u), M(v)) in H. This takes O(m'(n+m)). Traversing all edges in G takes O(n+m), and checking if an edge exists in H is O(m'), where m' is the number of edges in H.

Reduction

Input Transformation

Starting from 3SAT
3SAT (or any variant of SAT without the word Exact in it) specifies at most three literals. Remember to account for 1 or 2 literals per clause!

Making Clique
Either just make a clique (i.e., take k vertices and fully connect them), or add the edges of a graph G (E) to the edges of its complement (E'). The latter gives a fully connected graph which itself is a maximal clique of size |V|.

Adding a Gadget
Problems like this include Flower and Kite, where a common but unique subgraph (called a gadget) is added to all vertices. For example, we add a unique stem to all possible vertices as the input transformation to the Flower problem. Specifying that it is one stem per vertex is important, and not a singular stem that connects to all vertices -- the graders are primed to interpret ambiguity in the latter (incorrect) way.

Output Transformation

Note that the runtime **always depends on the original input to the known problem**. Do not modify the runtime such that it conforms to the output of the unknown problem.

Proof of Correctness

I think there are multiple "types" of reductions and their associated proofs.

1. For reductions that involve generalisation, the proof of correctness is weird because both are technically the same type of problem. It might be helpful to state that "we have created an equivalent case...", so if a solution exists/not exists for A, the solution must also exist/not exist for B. This is a non-statement to me but the sample answers seem to use it.
2. For reductions that involve modifications, a valid tactic is to show how the modifications cannot possibly create a solution for the unknown problem.
3. For reductions that involve more modifications (i.e., a different type of problem), it might be helpful to think of it as:
    a. A solution exists in the input for A iff a solution exists in the input provided to B. These two inputs are not necessarily equivalent.
    b. For example, a dominating set of size k exists in G' iff a vertex cover of size k exists in G, because the modifications done to G guarantee that…